

RAPPORT SYSTEM

```
Usage of /: 0.2% of 1006.85GB    Users logged in: 0
Memory usage: 9%                IPv4 address for eth0: 172.25
Swap usage: 0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn more
at https://ubuntu.com/engage/secure-kubernetes-at-the-edge
roK8s
just raised the bar for easy, resilient and secure K8s cluster
management.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/nermine/.hushlogin file.
nermine@DESKTOP-0112FBV:~$
```

KIROUANE NERMINE

2026/04/24

SYSTEM

INTRODUCTION

this tp is about learning to create processes and observe them

Exercise 1

1. Utilisez le manuel (commande man) pour renseigner les commandes: bg, fg, jobs, nohup, ps, top, pstree, kill, killall, trap.

```
TOP(1)                                User Commands                                TOP(1)

NAME
    top - display Linux processes

SYNOPSIS
    top [options]

DESCRIPTION
    The top program provides a dynamic real-time view of a running
    system. It can display system summary information as well as a
    list of processes or threads currently being managed by the
```

finding what each of the command do using man or help

```
NOHUP(1)                              User Commands                              NOHUP(1)

NAME
    nohup - run a command immune to hangups, with output to a non-
    tty

SYNOPSIS
    nohup COMMAND [ARG]...
    nohup OPTION

DESCRIPTION
    Run COMMAND, ignoring hangup signals.
```

PS(1)

User Commands

PS(1)

NAME

`ps` - report a snapshot of the current processes.

SYNOPSIS

`ps` [options]

DESCRIPTION

`ps` displays information about a selection of the active processes. If you want a repetitive update of the selection and the displayed information, use `top` instead.

This version of `ps` accepts several kinds of options:

1 UNIX options, which may be grouped and must be preceded by

PSTREE(1)

User Commands

PSTR

NAME

`pstree` - display a tree of processes

SYNOPSIS

`pstree` [-a, --arguments] [-c, --compact-not] [-C, --color
[-g, --show-pgids] [-h, --highlight-all, -H pid, --high-
light-pid pid] [-l, --long] [-n, --numeric-sort]
[-N, --ns-sort ns] [-p, --show-pids] [-s, --show-parents]
[-S, --ns-changes] [-t, --thread-names] [-T, --hide-thread
[-u, --uid-changes] [-Z, --security-context]
[-A, --ascii, -G, --vt100, -U, --unicode] [pid, user]
`pstree` -V, --version

DESCRIPTION

```

KILL(1)                                User Commands                                KILL

NAME
    kill - send a signal to a process

SYNOPSIS
    kill [options] <pid> [...]

DESCRIPTION
    The default signal for kill is TERM. Use -l or -L to list
    available signals. Particularly useful signals include HUP,
    INT, KILL, STOP, CONT, and 0. Alternate signals may be spe-
    cified in three ways: -9, -SIGKILL, or -KILL. Negative PID val-

```

```

KILLALL(1)                             User Commands                             KILLALL(1)

NAME
    killall - kill processes by name

SYNOPSIS
    killall [-Z, --context pattern] [-e, --exact]
    [-g, --process-group] [-i, --interactive] [-n, --ns PID]
    [-o, --older-than TIME] [-q, --quiet] [-r, --regexp]
    [-s, --signal SIGNAL, -SIGNAL] [-u, --user user] [-v, --ver-
   bose] [-w, --wait] [-y, --younger-than TIME] [-I, --ignore-
    case] [-V, --version] [--] name ...
    killall -l, --list
    killall -V, --version

DESCRIPTION
    killall sends a signal to all processes running any of the

```

```

no manual entry for trap
nermine@DESKTOP-0112FBV:~$ help trap
trap: trap [-lp] [[arg] signal_spec ...]
    Trap signals and other events.

```

```

given.
nermine@DESKTOP-0112FBV:~$ help bg
bg: bg [job_spec ...]
    Move jobs to the background.

```

```
nermine@DESKTOP-0112FBV:~$ help fg
fg: fg [job_spec]
    Move job to the foreground.
```

```
nermine@DESKTOP-0112FBV:~$ help jobs
jobs: jobs [-lnprs] [jobspec ...] or jobs -x command [args]
    Display status of jobs.
```

here we can see that:

bg : used to move jobs (processus)to the background

fg:move job to the foreground

trap:to catch signal (was is ctr+z or ctrl + C or closing terminal

nohup:runn the pgm even if terminal is closed

jobs:display jobs status

ps: get pid and ppid of the programs

ps tree: use to display processus as tree(to know lesfils and the père)

```
nermine@DESKTOP-0112FBV:~$ pstree
systemd--2*[agetty]
      |--cron
      |--dbus-daemon
      |--init-systemd(Ub)
      |   |--SessionLeader--Relay(287)--bash--pstr
      |   |--init--{init}
      |   |--login--bash
      |   |--{init-systemd(Ub)}
      |--rsyslogd--3*[{rsyslogd}]
      |--systemd--(sd-pam)
      |--systemd-journal
      |--systemd-logind
      |--systemd-resolve
      |--systemd-timesyn--{systemd-timesyn}
      |--systemd-udev
      |--unattended-upgr--{unattended-upgr}
      |--wsl-pro-service--7*[{wsl-pro-service}]
nermine@DESKTOP-0112FBV:~$ |
```

2. Créez un programme en langage C qui fait une boucle vide infinie et lancez le processus correspondant.

```
GNU nano 7.2
#include <stdio.h>

int main(){
while (1){
printf("hello");
}
return 0;
}
```

we created the program that will run hello for ever

```
nermine@DESKTOP-0112FBV:~$ ./hell.c
-bash: ./hell.c: No such file or directory
nermine@DESKTOP-0112FBV:~$ ./hell
```

start running it

```
llohellohellohellohellohellohell
ohellohellohellohellohellohello
ellohellohellohellohellohellohel
lohellohellohellohellohellohello
hellohellohellohellohellohellohe
llohellohellohellohellohellohell
ohellohellohellohellohellohello
ellohellohellohellohellohellohel
lohellohellohellohellohellohello
hellohellohellohellohellohellohe
llohellohellohellohellohellohell
ohellohellohellohellohellohello
ellohellohellohellohellohellohel
lohellohellohellohellohellohello
hellohellohellohellohellohellohe
llohellohellohellohellohellohell
ohellohellohellohellohellohello
ellohellohellohellohellohellohel
lohellohellohellohellohellohello
hellohellohellohellohellohellohe
llohellohellohellohellohellohell
ohellohellohellohellohellohello
ellohellohellohellohellohellohel
lohellohellohellohellohellohello
hellohellohellohellohellohellohe
```

run forever (while(true))

3. Donnez son pid et le pid de son processus père. (Utilisez la commande ps)

```
lohellohellohellohe^Z
[1]+  Stopped                  ./hell
hermine@DESKTOP-0112FBV:~$ ps -l

```

	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	1000	287	286	0	80	0	-	1550	do_wai	pts/0	00:00:00	bash
0	T	1000	712	287	20	80	0	-	670	do_sig	pts/0	00:00:00	hell
0	R	1000	713	287	0	80	0	-	2078	-	pts/0	00:00:00	ps

```
hermine@DESKTOP-0112FBV:~$ |
```

i found information about processus

pid of the program running is (hell.c)

we can see

pid of hell.c is 712

ppid of hell.c is bash with pid 287

4. Que se passera-t-il lorsque vous pressez la combinaison de touches CTRL+Z ?

```
lohellohellohellohe^Z
[1]+  Stopped                  ./hell
hermine@DESKTOP-0112FBV:~$
```

the program stop but not get killed

so it only suspend the program the program existe

5. Exécutez la commande ps à nouveau. Dans quel état il se trouve le processus lancé ?

```
hermine@DESKTOP-0112FBV:~$ ps -l

```

	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	1000	287	286	0	80	0	-	1550	do_wai	pts/0	00:00:00	bash
0	T	1000	712	287	20	80	0	-	670	do_sig	pts/0	00:00:00	hell
0	R	1000	713	287	0	80	0	-	2078	-	pts/0	00:00:00	ps

```
hermine@DESKTOP-0112FBV:~$
```

we can see the state of the programs

hell is : T (Stopped)

ps is : R (Running)

bash is : S (sleeping)

6. Comment remettre un processus au premier plan après un CTRL+Z ?

as we know fg bring back the program so



IT worked


```
nermine@DESKTOP-0112FBV: X + v
lohellohellohellohellohellohellohellohellohellohello
ellohellohellohellohellohellohellohellohellohellohel
ohellohellohellohellohellohellohellohellohellohello
llohellohellohellohellohellohellohellohellohellohell
hellohellohellohellohellohellohellohellohellohellohe
lohellohellohellohellohellohellohellohellohellohello
ellohellohellohellohellohellohellohellohellohellohel
ohellohellohellohellohellohellohellohellohellohello
llohellohellohellohellohellohellohellohellohellohell
hellohellohellohellohellohellohellohellohellohellohe
lohellohellohellohellohellohellohellohellohellohello
ellohellohellohellohellohellohellohellohellohellohel
ohellohellohellohellohellohellohellohellohellohello
llohellohellohellohellohellohellohellohellohellohell
hellohellohellohellohellohellohellohellohellohellohe
lohellohellohellohellohellohellohellohellohellohello
ellohellohellohellohellohellohellohellohellohellohel
ohellohellohellohellohellohellohellohellohellohello
```

run the program

```
lohellohellohellohellohellohellohellohellohellohello
hellohellohellohellohellohellohellohellohellohellohe^Z
[1]+  Stopped                  ./hell
nermine@DESKTOP-0112FBV:~$ ps -l
F S  UID      PID      PPID  C  PRI   NI  ADDR  SZ  WCHAN  TTY      00:0
D
4 S  1000      764      759   0   80    0   -    1517 do_wai pts/2      00:0
sh
0 T  1000      830      764  22   80    0   -     670 do_sig pts/2      00:0
ll
0 R  1000      831      764   0   80    0   -    2078 -      pts/2      00:0
nermine@DESKTOP-0112FBV:~$ |
```

```
lohellohellohellohellohellohellohellohellohellohello
ohellohellohellohellohellohellohellohellohellohello
ellohellohellohellohellohellohellohellohellohellohell
lohellohellohellohellohellohellohellohellohellohello
ohellohellohellohellohellohellohellohellohellohello
- Stopped                  ./hell
nermine@DESKTOP-0112FBV:~$ |
nermine@DESKTOP-0112FBV:~$: command not found
nermine@DESKTOP-0112FBV:~$ kill -SIGTSTP 871
nermine@DESKTOP-0112FBV:~$
```

the program is stopped

```
nermine@DESKTOP-0112FBV:~$: command not found
^C
-bash:: command not found
nermine@DESKTOP-0112FBV:~$: command not found
nermine@DESKTOP-0112FBV:~$ kill -SIGTSTP 871
nermine@DESKTOP-0112FBV:~$ ps -l
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCH
4	S	1000	804	798	0	80	0	-	1550	do_
wai		pts/0	00:00:00	bash						
0	R	1000	935	804	0	80	0	-	2078	-

same as using CTRL+Z

SIGTSTP = Signal Terminal Stop

7. Pressez la combinaison de touches CTRL+C. Que se passera-t-il ?

```
ellohellohellohellohellohellohellohellohe pts/0 00:00:00 ps
lohellohellohellohellohellohellohellohell nermine@DESKTOP-0112FBV:~$ kill -SIGINT 871
hellohellohellohellohellohellohellohell nermine@DESKTOP-0112FBV:~$ jobs -l
llohellohellohellohellohellohellohell nermine@DESKTOP-0112FBV:~$ jobs
nermine@DESKTOP-0112FBV:~$ | nermine@DESKTOP-0112FBV:~$ kill -SIGINT 942
nermine@DESKTOP-0112FBV:~$ |
```

the program is killed

SIGINT : Signal Interrupt

9. Comment lancer un processus en arrière plan ? Quelle est la différence avec la combinaison CTRL+Z ?

bg will make the program run still run but in the background

10. Quelle est la commande à taper pour afficher la liste des processus se trouvant en arrière plan ?

jobs

```
hermine@DESKTOP-0112FBV:~$ help jobs
jobs: jobs [-lnprs] [jobspec ...] or jobs -x command [a
Display status of jobs.

Lists the active jobs. JOBSPEC restricts output to
Without options, the status of all active jobs is c

Options:
  -l      lists process IDs in addition to the n
  -n      lists only processes that have changed
last
      notification
  -p      lists process IDs only
  -r      restrict output to running jobs
  -s      restrict output to stopped jobs

If -x is supplied, COMMAND is run after all job sp
appear in ARGS have been replaced with the process this
```

command jobs is the one that show processus running in the bg

Exercice 2

Soit le code suivant écrit en C :

```

#include <stdio.h>
#include <unistd.h>

int main() {

    printf("L'identifiant du processus est %d\n", getpid());
    printf("L'identifiant du processus parent est %d\n", getppid());

    return 0;
}

```

1. Exécutez ce programme plusieurs fois et noter les valeurs affichées ? Avez-vous les mêmes

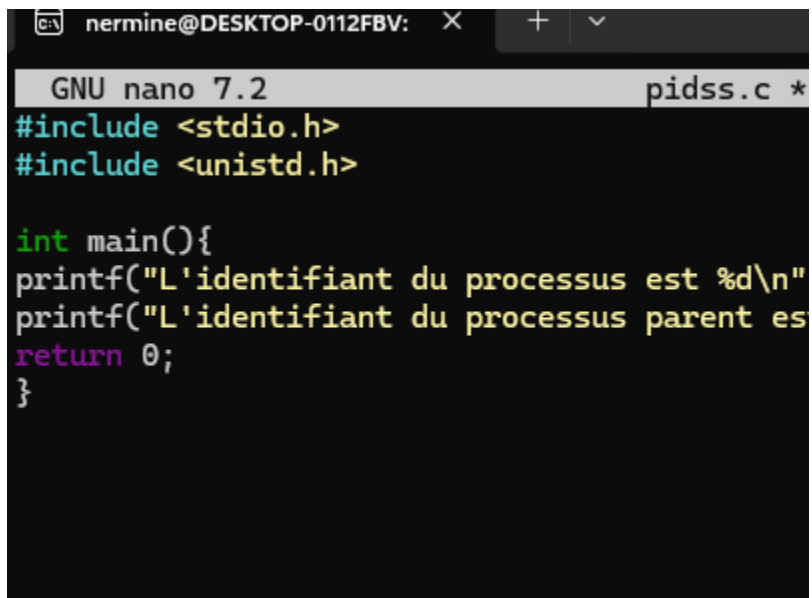
valeurs pour le pid et le ppid ? Expliquez.

```

    If -x is used, returns the exit status of COMMAND.
nermine@DESKTOP-0112FBV:~$ touch pidss.c
nermine@DESKTOP-0112FBV:~$ nano pidss.c

```

creating the program



```

nermine@DESKTOP-0112FBV: X + v
GNU nano 7.2 pidss.c *
#include <stdio.h>
#include <unistd.h>

int main(){
printf("L'identifiant du processus est %d\n"
printf("L'identifiant du processus parent es
return 0;
}

```

```

nermine@DESKTOP-0112FBV:~$ nano pidss.c
nermine@DESKTOP-0112FBV:~$ gcc pidss.c -o pidss
nermine@DESKTOP-0112FBV:~$ ./pidss
L'identifiant du processus est 1169
L'identifiant du processus parent est 1110
nermine@DESKTOP-0112FBV:~$

```

run the program

```

nermine@DESKTOP-0112FBV:~$ ./pidss
L'identifiant du processus est 1169
L'identifiant du processus parent est 1110
nermine@DESKTOP-0112FBV:~$ ps -l
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  T
D
4 S  1000      1110      1109  0   80   0  -   1550 do_wai p

```

```

nermine@DESKTOP-0112FBV:~$ gcc pidss.c -o pidss
nermine@DESKTOP-0112FBV:~$ ./pidss
L'identifiant du processus est 1169
L'identifiant du processus parent est 1110
nermine@DESKTOP-0112FBV:~$ ps -l
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  T
D
4 S  1000      1110      1109  0   80   0  -   1550 do_wai p
sh
0 R  1000      1172      1110 99   80   0  -   2078 -        p
nermine@DESKTOP-0112FBV:~$ ./pidss
L'identifiant du processus est 1173
L'identifiant du processus parent est 1110
nermine@DESKTOP-0112FBV:~$ ./pidss
L'identifiant du processus est 1174
L'identifiant du processus parent est 1110
nermine@DESKTOP-0112FBV:~$

```

the pid keep changing (id of processus fil keep changing)

the ppid do not change because the processus père is still always bash while each time we are running new program so pid will change for sure because it a new program even it the same program but when we run it we are running a new one each time

